EL2805 Reinforcement Learning

Exercise Session 5

November 28th 2025

Division of Decision and Control Systems
School of Electrical Engineering and Computer Science
KTH Royal Institute of Technology

Instructions.

- These exercises are for your practice, and you do not need to submit solutions. We encourage you to work on them before the corresponding exercise session on Dec 5th. The materials for the 6th exercise session will be available on Dec 8th.
- In each exercise session, we will go through a selection of topics chosen by the teaching assistant and participating students.
- Homework and labs will be based on concepts covered in lectures and exercise sessions. These materials are excellent preparation for your graded assignments!
- There is no need to memorize equations - relevant materials will be provided during the exam. Focus on concepts and understanding!
- The materials cover a wide range of topics, so feel free to focus on what interests you most. However, to develop a well-rounded understanding of RL, it is important that you engage with both theoretical and practical exercises.
- We use green boxes to highlight established results and definitions, yellow for problems that require your solutions, and pink for high-level remarks. Exercises marked with a ★ are more challenging and intended for students seeking an extra challenge.
- We value your feedback on the materials provided. If you have any questions, concerns, or suggestions, do not hesitate to reach out to any of the TAs.

Soft Actor-Critic (SAC)

Off-Policy Maximum Entropy Deep RL with a Stochastic Actor

In this session, we will explore actor-critic methods, focusing on one of the most widely used algorithms: Soft Actor-Critic (SAC) [1]. Actor-critic methods generalize the idea of Policy Iteration (PI), introduced in Exercise Session 1. Broadly, the critic is a neural network that approximates the policy evaluation step of PI, while the actor performs the policy improvement step.

Actor-critic methods are often seen as bridging the gap between value-based methods (ex. Q-learning and DQN) and policy-gradient methods (ex. REINFORCE). In Exercise Session 4, we studied how REINFORCE updates parameter using the following gradient:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau\sim\pi_\theta}\left[\left(\sum_{t=1}^{T}\nabla_\theta \log \pi_\theta(a_t|s_t)\right)\left(\sum_{t=1}^{T} r_t\right)\right] \qquad \text{(REINFORCE gradient)}$$

This requires waiting until the end of an episode ($t = T$) to perform the update. Lecture 6 presented a similar gradient expression for the discounted (infinite-horizon) case:

$$\nabla_\theta J(\theta) = \frac{1}{1-\gamma}\mathbb{E}_{\substack{s\sim d^{\pi_\theta}\\ a\sim\pi_\theta(\cdot|s)}}\left[\nabla_\theta \log \pi_\theta(a|s) A^{\pi_\theta}(s,a)\right] \tag{1}$$

In the previous session, we also discussed how TRPO updates policy parameters θ_+ based on the current θ by optimizing:

$$\max_{\theta_+}\mathbb{E}_{\substack{s\sim d^{\pi_\theta}\\ a\sim\pi_\theta(\cdot|s)}}\left[\frac{\pi_{\theta_+}(a|s)}{\pi_\theta(a|s)} A^{\pi_\theta}(s,a)\right] \qquad \text{(TRPO update)}$$

under the constraint $\mathbb{E}_{s\sim d^{\pi_\theta}}\left[D_{\mathrm{KL}}(\pi_\theta(\cdot|s), \pi_{\theta_+}(\cdot|s))\right] \leq \delta$.

Both of these methods rely on an estimate of $A^{\pi_\theta}(s,a)$, which we have yet to address fully. This session introduces actor-critic algorithms to solve this challenge.

1.1 Preliminary Algorithm: Advantage Actor-Critic (A2C)

To estimate $A^{\pi_\theta}(s,a)$, recall from the previous exercise session (Exercise 2.1, step 2) that:

$$A^{\pi_\theta}(s,a) = \mathbb{E}_{s'\sim P(\cdot|s,a)}[r(s,a) + \gamma V^{\pi_\theta}(s') - V^{\pi_\theta}(s)].$$

Sampling gives us the approximation: $A^{\pi_\theta}(s_t,a_t) = r(s_t,a_t) + \gamma V^{\pi_\theta}(s_{t+1}) - V^{\pi_\theta}(s_t)$. We represent V^{π_θ} using a neural network with parameters ϕ ($V_\phi \approx V^{\pi_\theta}$). The parameters ϕ are trained by minimizing the following loss over a batch B:

$$J(\phi) = \frac{1}{|B|}\sum_{(s_t,a_t,s_{t+1})\in B}\Big(r(s_t,a_t) + \gamma V_\phi(s_{t+1}) - V_\phi(s_t)\Big),$$

and then updated as $\phi \leftarrow \phi - \alpha\nabla_\phi J(\phi)$. This network, with parameters ϕ, is called the **critic** (it evaluates/criticizes current policy). In contrast, network π_θ, called the **actor** (it acts, decides next action), is updated using (1), replacing A^{π_θ} with the critic's estimate A_ϕ.

This approach forms the Advantage Actor-Critic (A2C) algorithm (recall that A^π is called advantage function). Actor-critic methods require both value estimation (for critic update, V_ϕ), and policy gradients (for actor update, π_θ).

So far, we have studied multiple policy-gradient based algorithms: REINFORCE, DDPG, TRPO, and now A2C. To clarify their differences, complete the following exercise:

Exercise 1. Choose the correct algorithm (REINFORCE/DDPG/TRPO/A2C) for each column title and fill in the missing entries:

Algorithm				
On/Off-Policy	On-policy	On-policy		Off-policy
Action Space		Discrete/ Continuous		
Sample Efficiency			Low	High
Exploration		Implicit		Additive noise on actions
Stability	High		Low (high variance)	
Biggest disadvantage	Computationally expensive			

1.2 Maximizing Entropy for Exploration

As seen in the table above, many methods employ implicit exploration. However, there are scenarios where we may wish to explicitly control or amplify the level of exploration. Next, we introduce a method for achieving this control.

Consider a given policy π, where $\pi(a|s)$ denotes the probability of selecting action a given state s. The **entropy** of policy π for state s is defined as:

$$\mathcal{H}(\pi(\cdot|s)) = \mathbf{E}_{a\sim\pi(\cdot|s)} \log\left(\frac{1}{\pi(a|s)}\right) = \sum_a \pi(a|s) \log\left(\frac{1}{\pi(a|s)}\right).$$

In analogy to physics, entropy quantifies uncertainty. Probability distributions $\pi(\cdot|s)$ that approximate a uniform distribution (i.e. $\pi(a_1|s) \approx \pi(a_2|s)$ for any a_1, a_2) exhibit higher entropy. Intuitively, increasing the entropy of a policy π makes it more uniform-like, thereby encouraging greater exploration. Next, we prove this more precisely.

To facilitate exploration, we introduce soft-value functions $V^\pi_\mathcal{H}$ and $Q^\pi_\mathcal{H}$ as extensions of the standard value functions V^π and Q^π:

$$V^\pi_\mathcal{H}(s) = \mathbb{E}_{a\sim\pi(\cdot|s)}[Q^\pi_\mathcal{H}(s,a)] + \mathcal{H}(\pi(\cdot|s)) \tag{2}$$
$$Q^\pi_\mathcal{H}(s,a) = r(s,a) + \gamma \mathbb{E}_{s'\sim P(\cdot|s,a)} V^\pi_\mathcal{H}(s') \tag{3}$$

Exercise 2. To develop a better understanding of soft-value functions, solve:

1. Show that under a deterministic policy π, $V^\pi_\mathcal{H}$ reduces to standard values V^π.
2. Let $\pi(\cdot|s)$ be ε-greedy policy. If, for all s, a, $r(s,a) = R = \text{const}$, identify an optimal ε that maximizes $V^\pi_\mathcal{H}(s)$.
3. Derive a reward function $r^\pi_\mathcal{H}$ such that $V^\pi_\mathcal{H}$ and $Q^\pi_\mathcal{H}$ satisfy the standard equations of V^π and Q^π.

From the last point of the exercise, we conclude that since soft-value functions satisfy the same equations as standard value functions (albeit with a modified reward function), results like the convergence of policy evaluation still hold. In other words, starting with an arbitrary estimate V for $V^\pi_\mathcal{H}$ and iteratively applying (3) and (2), we are guaranteed to converge to $Q^\pi_\mathcal{H}$, similar to classical policy evaluation.

Next, we derive an analogous policy improvement step for soft-value functions. Define a new policy π_{new} based on the previous policy π_{old} as:

$$\pi_{\text{new}} = \arg\min_{\pi' \in \Pi} D_{\text{KL}} \left(\pi'(\cdot|s_t) \middle|\middle| \frac{\exp(Q_{\mathcal{H}}^{\pi_{\text{old}}}(s_t, \cdot))}{Z^{\pi_{\text{old}}}(s_t)} \right)^{1}$$

where the optimization is over the set of policies Π (ex. those parametrized by a fixed neural network architecture, $\pi_{\text{old}} \in \Pi$), and $Z^{\pi_{\text{old}}}$ is the partition function that serves as normalization (**independent** of π'!). Then we ask you to show the following:

★ **Exercise 3.** Show that the policy π_{new} improves upon π_{old} by demonstrating:

$$Q_{\mathcal{H}}^{\pi_{\text{old}}}(s, a) \leq Q_{\mathcal{H}}^{\pi_{\text{new}}}(s, a), \quad \forall s, a.$$

⚠Hints

1. First use that π_{new} minimizes D_{KL}, and thus has smaller D_{KL} than $\pi' = \pi_{\text{old}}$, to show:

$$V_{\mathcal{H}}^{\pi_{\text{old}}}(s) \leq \mathbb{E}_{a \sim \pi_{\text{new}}(\cdot|s)} \left[Q_{\mathcal{H}}^{\pi_{\text{old}}}(s, a) - \log \pi_{\text{new}}(a|s) \right].$$

2. Then use definition of $Q_{\mathcal{H}}^{\pi_{\text{old}}}(s, a)$ and upper bound $V_{\mathcal{H}}^{\pi_{\text{old}}}(s')$ using 1. Repeat iteratively.

These results indicate that we can iteratively update soft-value functions using the same principles as in standard policy iteration algorithm, while ensuring convergence to the optimal soft-policy. Furthermore, by adjusting the entropy term in the soft-value function, we gain direct control over the level of exploration in the learning process.

1.3 Trick the Gradient!

Before we introduce the Soft Actor-Critic (SAC) algorithm, we need to present one more technical trick. Note, however, that this method can be applied to general RL algorithms and is not specific to SAC. We begin by recalling from the previous Exercise Session that we estimated policy gradients for the REINFORCE algorithm using the fact that $\nabla_\theta \log \pi_\theta(a|s) = \frac{\nabla_\theta \pi_\theta(a|s)}{\pi_\theta(a|s)}$. To better understand what we are doing, let's consider a more general scenario.

Let p_θ be a discrete probability distribution, x a random variable with this distribution, and f a function defined on x. Then:

$$\nabla_\theta \mathbb{E}_{x \sim p_\theta}[f(x)] = \nabla_\theta \sum_x p_\theta(x) f(x) = \sum_x p_\theta(x) \frac{\nabla_\theta p_\theta(x)}{p_\theta(x)} f(x) = \sum_x p_\theta(x) \nabla_\theta \log p_\theta(x) f(x)$$

$$= \mathbb{E}_{x \sim p_\theta}[\nabla_\theta \log p_\theta(x) f(x)] \approx \frac{1}{N} \sum_{i=1}^{N} \nabla_\theta \log p_\theta(x_i) f(x_i)$$

In the context of REINFORCE, p_θ corresponds to π_θ, x corresponds to the chosen action a, and f corresponds to $\sum_{t=1}^{T} r(s_t, a_t)$. At a high level, the challenge is that we wish to compute the gradient with respect to parameters defining the distribution from which random variables are sampled. We address this challenge using a derivative trick that introduces the gradient inside the expectation, allowing approximation via samples.

Reparameterization Trick. Next, we introduce another method to approximate gradients. This methods has stricter requirements but typically results in estimates with lower variance. Suppose we can express the random variable x as a transformation of a standard Gaussian random variable $\varepsilon \sim \mathcal{N}(0, 1)$ (ex. if $x \sim \mathcal{N}(\mu, \sigma^2)$ then $x = \sigma\varepsilon + \mu$). Let this transformation be g_θ (note that the distribution p_θ of x depends on θ, while distribution of ε does not). Then:

$$\nabla_\theta \mathbb{E}_{x \sim p_\theta}[f(x)] = \nabla_\theta \mathbb{E}_{\varepsilon \sim \mathcal{N}(0,1)}[f(g_\theta(\varepsilon))] = \mathbb{E}_{\varepsilon \sim \mathcal{N}(0,1)}[\nabla_\theta f(g_\theta(\varepsilon))] \approx \frac{1}{N} \sum_{i=1}^{N} \nabla_\theta f(g_\theta(\varepsilon_i))$$

[1] Recall that KL-divergence between probability distributions $p(x)$ and $q(x)$ is $D_{\text{KL}}(p||q) = \sum_x p(x) \log \frac{p(x)}{q(x)}$.

Thus, both $\frac{1}{N}\sum_{i=1}^{N}\nabla_\theta \log p_\theta(x_i) f(x_i)$ (from REINFORCE) and $\frac{1}{N}\sum_{i=1}^{N}\nabla_\theta f(g_\theta(\varepsilon_i))$ (from reparameterization trick) are estimates of $\nabla_\theta \mathbb{E}_{x\sim p_\theta}[f(x)]$. We use this for the soft-value functions:

Exercise 4. (Reparameterization trick) Let the loss function be defined as:

$$J(\theta) = \mathbb{E}_{s\sim\mathcal{D}}\left[D_{\mathrm{KL}}\left(\pi_\theta(\cdot|s)\middle\|\frac{\exp(Q_{\mathcal{H},\phi}(s,\cdot))}{Z_\phi(s)}\right)\right],$$

and assume that there exists a function g_θ such that for $\varepsilon\sim\mathcal{N}(0,1)$ and $a\sim\pi_\theta(\cdot|s)$, we have $a = g_\theta(\varepsilon, s)$. First, show that:

$$J(\theta) = \mathbb{E}_{s\sim\mathcal{D},\varepsilon\sim\mathcal{N}(0,1)}\Big[\log\pi_\theta(g_\theta(\varepsilon,s)|s) - Q_{\mathcal{H},\phi}(s, g_\theta(\varepsilon,s))\Big] + \text{const}$$

where const is with respect to θ. Then, derive $\nabla_\theta J(\theta)$. What functions must be differentiable for this method to work? What requirements are necessary for REINFORCE-like gradient estimation? What are the benefits of using this method?

⚠Hints

Recall the chain rule for differentiable functions f, u, v:

$$\frac{d}{dx}f(u(x),v(x)) = \frac{\partial u}{\partial x}\frac{\partial f}{\partial u}\Big|_{u(x)} + \frac{\partial v}{\partial x}\frac{\partial f}{\partial v}\Big|_{v(x)}.$$

1.4 Deciphering SAC

In this section, we delve into the specifics of the SAC algorithm. Suppose you decide to use an SAC implementation from OpenAI. Before proceeding, it's essential to understand their approach. You copy the pseudo-code from `https://spinningup.openai.com/en/latest/algorithms/sac.html` to Algorithm 1 on the next page.

Take some time to analyze the different parts of the pseudo-code. Refer to the previous exercises in this session as well as the results from earlier sessions. Once you feel ready, try to solve the following exercise:

Exercise 5. Answer the following questions:

1. Is SAC an off- or on-policy algorithm? Why?
2. Where does SAC perform exploration?
3. Does this algorithm works for discrete/continuous action spaces?
4. Note that actions $\tilde{a}'$ are not sampled from the buffer but from the current policy. Why is this the case?
5. One possible parameterization of policy π_θ is using squashing function $\tilde{a}_\theta(s) = \tanh(\mu_\theta(s)+\sigma_\theta(s)\odot\varepsilon)$ with $\varepsilon\sim\mathcal{N}(0,I)$. Explain logic behind such parameterization.
6. What would happen if the target networks were updated as $\phi_{\text{target},\ell}\leftarrow\phi_\ell$ (with $\rho = 0$)?
7. Why are two target networks necessary in SAC?
8. What does the parameter α represent? Which algorithm is closest to SAC when $\alpha = 0$, and why?
9. Fill in the table for SAC. Do all the terms in the subtitle (Off-Policy Maximum Entropy Deep RL with a Stochastic Actor) make sense?

References

[1] Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *International conference on machine learning*, pages 1861–1870. PMLR, 2018.

Algorithm 1 Soft Actor-Critic

Require: Initial policy parameters θ, Q-function parameters ϕ_1, ϕ_2, empty replay buffer $\mathcal{D}$

Set target parameters equal to main parameters $\phi_{\text{target},1} \leftarrow \phi_1, \phi_{\text{target},2} \leftarrow \phi_2$
repeat
  Observe state s and select action $a \sim \pi_\theta(\cdot|s)$
  Execute a in the environment
  Observe next state s', reward r, and done signal d to indicate whether s' is terminal
  Store (s, a, r, s', d) in replay buffer $\mathcal{D}$
  if s' is terminal **then**
    Reset environment state
  end if
  if it's time to update then **then**
    for j in range (however many updates) **do**
      Randomly sample a batch of transitions, $B = \{(s, a, r, s', d)\}$ from $\mathcal{D}$
      Compute targets for the Q-functions:

$$y(r, s', d) = r + \gamma(1 - d)\left(\min_{\ell=1,2} Q_{\phi_{\text{target},\ell}}(s', \tilde{a}'_\theta) - \alpha \log \pi_\theta(\tilde{a}'_\theta|s')\right), \quad \tilde{a}'_\theta \sim \pi_\theta(\cdot|s')$$

      Update Q-functions by one step of gradient descent using:

$$\nabla_{\phi_\ell} \frac{1}{|B|} \sum_{(s,a,r,s',d) \in B} \left(Q_{\phi_\ell}(s, a) - y(r, s', d)\right)^2 \quad \text{for } \ell = 1, 2$$

    end for
    Update policy by one step of gradient ascent using reparameterization trick and:

$$\nabla_\theta \frac{1}{|B|} \sum_{s \in B} \left(\min_{\ell=1,2} Q_{\phi_\ell}(s, \tilde{a}_\theta(s)) - \alpha \log \pi_\theta(\tilde{a}_\theta(s)|s)\right), \quad \tilde{a}_\theta(s) \sim \pi_\theta(\cdot|s)$$

    Update target networks with:

$$\phi_{\text{target},\ell} \leftarrow \rho \phi_{\text{target},\ell} + (1 - \rho)\phi_\ell \quad \text{for } \ell = 1, 2$$

  end if
until convergence